



Módulo 8

Versionado, linaje y documentación

Curso: Data Engineer Básico

60 min

Objetivos de Aprendizaje

- Implementar control de versiones en scripts de ingestión y transformación utilizando Git
- Versionar esquemas de datos de forma que se mantenga trazabilidad entre cambios de estructura y pipelines
- Documentar el linaje de datos de un pipeline de obra lineal aplicando la plantilla estándar de metadatos
- Aplicar convenciones de commit que permitan identificar cambios en datos y esquemas de forma clara

1. Introducción al control de versiones para datos

10 min

¿Por qué versionar datos?

El versionado en un contexto de ingeniería de datos va más allá del código tradicional.

Reproducibilidad

Garantizar que un pipeline puede reconstruirse exactamente como fue ejecutado

Auditoría

Registrar quién cambió qué, cuándo y por qué

Colaboración

Facilitar el trabajo simultáneo en múltiples ramas de desarrollo

Rollback

Capacidad de revertir cambios problemáticos en datos o esquemas

Diferencias con el versionado de código tradicional

Aspecto	Código	Datos/Esquemas
Volumen	Kilobytes/Megabytes	Gigabytes/Terabytes
Historial	Completo	Parcial o muestreado
Diferencial	Líneas cambiadas	Registros agregados/modificados
Conflictos	Frecuentes	Menos frecuentes

Caso de uso: TransCore

En el proyecto TransCore, los scripts de ETL que procesan datos de obra lineal deben estar versionados para:

- Rastrear cambios en transformaciones que afectan **indicadores financieros**
- Mantener un historial de versiones de esquemas de tablas de **bronze, silver y gold**
- Documentar lineage cuando hay **cambios regulatorios** que requieren trazabilidad

Contexto: El equipo de Data Engineering necesita auditorías periódicas para cumplimiento SOX.

2. Git básico para Data Engineers

15 min

Configuración inicial

```
# Configurar identidad
git config --global user.name "Ingeniero de Datos"
git config --global user.email "ingeniero@transcore.com"

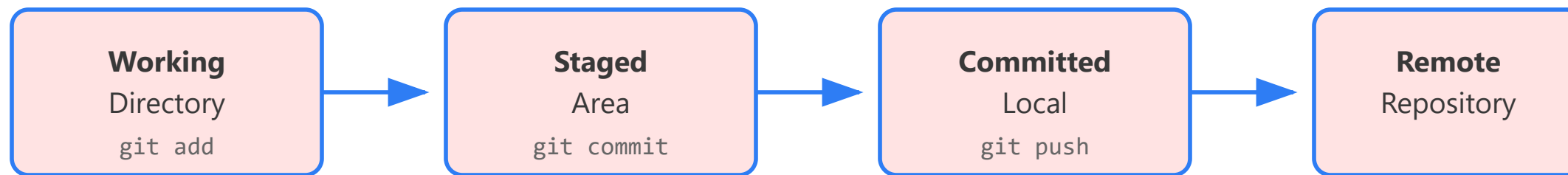
# Inicializar repositorio
git init

# Clonar repositorio existente
git clone https://gitlab.transcore.com/data/obra-lineal.git
```

Tip: Usa el email corporativo para que los commits estén asociados a tu identidad en la empresa.

Flujo de trabajo básico

Flujo de trabajo Git



Working Directory: Donde modificas los archivos

Staged Area: Archivos preparados para commit

Committed: Cambios guardados localmente

Remote: Repositorio centralizado (GitLab)

Comandos esenciales

```
# Ver estado del repositorio
git status

# Ver diferencias
git diff
git diff --staged

# Registrar cambios
git add <archivo>
git add .

# Crear commit con mensaje descriptivo
git commit -m "feat: actualiza esquema tabla silver_kpis_obra

- Agrega columna fecha_ejecucion
- Modifica tipo de cost_center de VARCHAR a INT
- Ref: JIRA-DE-2024"

# Ver historial
```

Ramas (Branches)

```
# Crear rama para desarrollo
git checkout -b feature/nuevo-indicador

# Listar ramas
git branch

# Cambiar de rama
git checkout main

# Fusionar cambios
git merge feature/nuevo-indicador
```

Error común: No hacer merge antes de empujar cambios. Siempre sincroniza con la rama principal antes de crear un PR.

Convenciones de commits para Data Engineering

```
<tipo>(<alcance>): <descripción>
```

```
[tipo]: feat | fix | docs | style | refactor | test | chore
```

```
[alcance]: ingestion | transformation | schema | pipeline
```

Ejemplos:

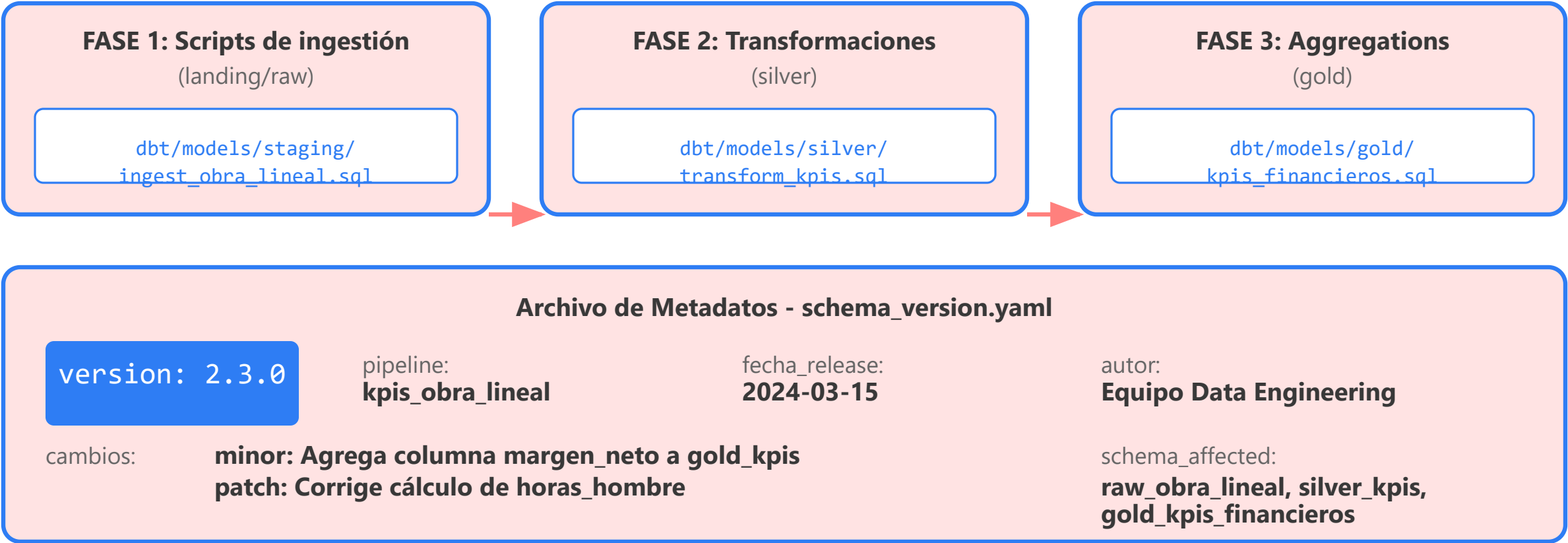
- `feat(ingestion):` agrega nuevo source de sensores IoT
- `fix(schema):` corrige tipo de dato en columna metros_totales
- `docs(pipeline):` actualiza documentación de linaje
- `refactor(transformation):` optimiza query de agregación gold

3. Versionado de esquemas y scripts

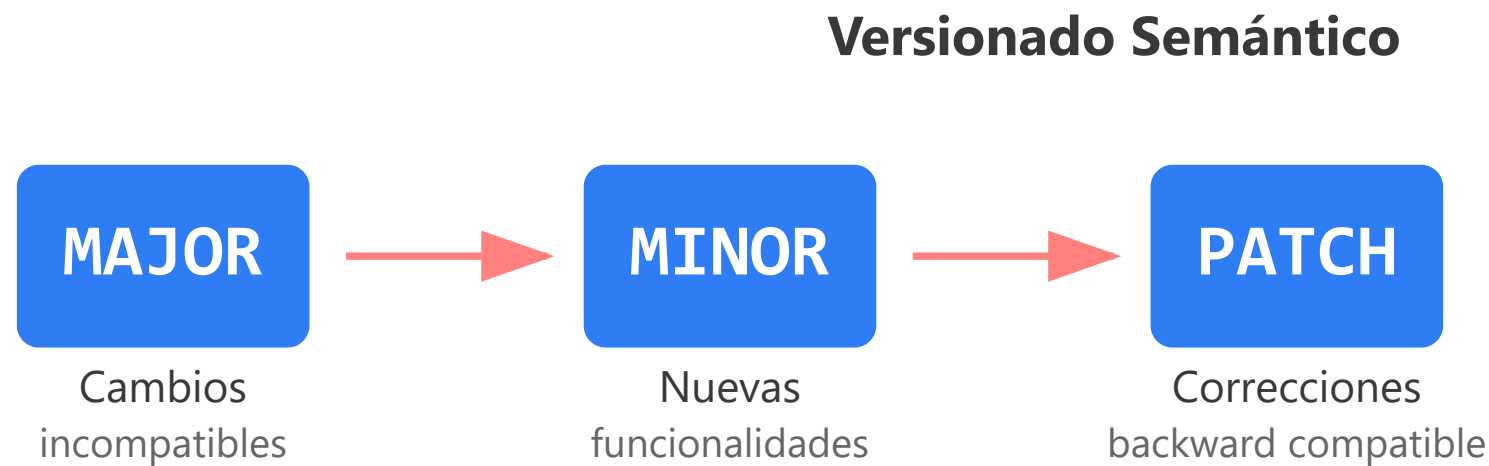
10 min

Estrategia de versionado para TransCore

Estrategia de Versionado - TransCore



Versionado semántico de esquemas



Ejemplos:

1.0.0 → 1.1.0 = Compatible
1.1.0 → 1.2.0 = Compatible
1.2.0 → 2.0.0 = BREAKING

Versión	Cambio	Impacto
1.0.0 → 1.1.0	Agregar columna opcional	Compatible
1.1.0 → 1.2.0	Agregar columna con valor por defecto	Compatible
1.2.0 → 2.0.0	Eliminar columna existente	Breaking change

Etiquetas (Tags) para releases de datos

```
# Etiquetar versión de pipeline para auditoría
git tag -a v2.3.0 -m "Release pipeline KPIs obra lineal - versión producción"

# Listar etiquetas
git tag

# Ver etiqueta específica
git show v2.3.0
```

Uso: Las etiquetas permiten identificar puntos específicos en el historial para auditorías y rollbacks.

Archivo de metadatos de esquema

```
# schema_version.yaml
version: "2.3.0"
fecha_release: "2024-03-15"
pipeline: "kpis_obra_lineal"
autor: "Equipo Data Engineering"
cambios:
  - tipo: "minor"
    descripcion: "Agrega columna margen_neto a gold_kpis"
    backward_compatible: true
  - tipo: "patch"
    descripcion: "Corrige cálculo de horas_hombre"
    backward_compatible: true
schema_affected:
  - "raw_obra_lineal"
  - "silver_kpis"
  - "gold_kpis_financieros"
```

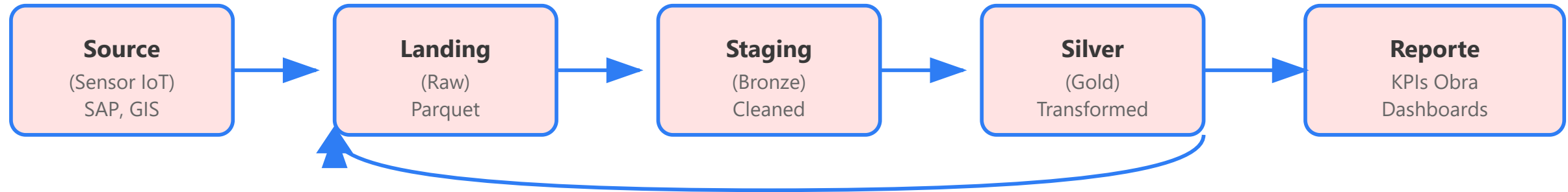
4. Linaje y metadatos: conceptos y estándares

10 min

¿Qué es el linaje de datos?

El **linaje de datos** (data lineage) describe el recorrido de los datos desde su origen hasta su destino final, incluyendo todas las transformaciones intermedias.

Linaje de datos - Pipeline Obra Lineal



Trazabilidad completa desde origen hasta consumo final

Beneficios del linaje de datos

Trazabilidad

Identificar de dónde vienen los datos en un reporte

Impact Analysis

Evaluar qué pasa si cambio una fuente

Debugging

Localizar dónde ocurrió un error de calidad

Cumplimiento regulatorio

GDPR, SOX, y otras normativas

Documentación automática

Reducir deuda técnica

Estándar OpenLineage

Estándar abierto de la Linux Foundation para capturar linaje en entornos de procesamiento de datos.

```
{
  "eventType": "COMPLETE",
  "eventTime": "2024-03-15T10:30:00Z",
  "run": { "runId": "run-12345" },
  "job": {
    "namespace": "transcore",
    "name": "pipeline_kpis_obra"
  },
  "inputs": [{
    "namespace": "transcore",
    "name": "silver_kpis_obra",
    "type": "TABLE"
  }],
  "outputs": [{
    "namespace": "transcore",
    "name": "gold_kpis_financieros",
    "type": "TABLE"
  }]
}
```

Metadatos de negocio

Campo	Descripción	Ejemplo
owner	Responsable del dataset	equipo.bi@transcore.com
department	Área de negocio	Finanzas
classification	Sensibilidad	Confidencial
refresh_rate	Frecuencia de actualización	Diario
quality_score	Puntuación de calidad	98.5%

Plantilla estándar de linaje para TransCore

REGISTRO DE LINAGE - OBRA LINEAL	
Identificador: [PL-2024-0034] Nombre: Pipeline KPIs Financieros Obra Lineal Versión: 2.3.0 Fecha: 2024-03-15	
NODOS DE ORIGEN — SAP ECC (VBAK, VBRP) — GIS (obras_lineales_gis)	
TRANSFORMACIONES — T01: Normalización de importes → transform_currency.sql — T02: Agregación por obra → aggregate_by_obra.sql — T03: Cálculo KPIs → calculate_kpis.sql	
NODOS DE DESTINO — gold_kpis_financieros (Aggregated) — gold_kpis_detalle (Detalle por línea)	

Lab: Ejercicio 3 – Registrar linaje en plantilla estándar

20 min

Información del Ejercicio

Duración	20 minutos
Tipo	Ejercicio práctico individual
Caso	TransCore - Pipeline de Obra Lineal

Archivos de trabajo

- source_sap_sample.csv
- source_gis_sample.csv
- pipeline_config.yaml

Objetivo

Aplicar los conceptos de linaje y versionado documentando un pipeline de ingestión y transformación de datos de obra lineal utilizando la plantilla estándar de TransCore.

Escenario

Eres parte del equipo de Data Engineering de TransCore. Se ha solicitado documentar el linaje completo del pipeline que genera los **KPIs financieros de obra lineal** para el reporte mensual a la dirección.

- **Source primario:** Sistema SAP (tablas VBAK, VBRP)
- **Source secundario:** Sistema GIS con información de obras
- **Procesamiento:** Ingesta → Landing → Silver → Gold
- **Frecuencia:** Diariamente a las 06:00 AM

Instrucciones

1. Analizar el dataset (5 min)

Revisar columnas, identificar relaciones, determinar transformaciones

2. Completar la plantilla de linaje (10 min)

Llenar nodos de origen, documentar transformaciones, especificar destino

3. Verificar consistencia (5 min)

Cada transformación con script, campos origen en destino final

Entregable

Documento: linaje_kpis_obra_lineal.md

- Plantilla de linaje completamente llena
- Lista de scripts del repositorio Git relacionados

Validación: Verificar que los campos origen aparecen en el destino final y que las versiones sean incrementales.

Resumen

Puntos Clave

- **Versionar datos** es fundamental para reproducibilidad, auditoría y colaboración
- **Git** es la herramienta estándar para control de versiones en Data Engineering
- **Convenciones de commits** (feat, fix, docs) facilitan el rastreo de cambios
- **Versionado semántico** (MAJOR.MINOR.PATCH) indica el tipo de cambio en esquemas
- **Linaje de datos** proporciona trazabilidad desde origen hasta consumo
- **OpenLineage** es un estándar abierto para capturar metadatos de linaje
- **Plantillas de metadatos** estandarizan la documentación de pipelines

Referencias

Recursos Adicionales

- [Git Documentation - Documentación oficial de Git](#)
- [OpenLineage - Estándar de linaje de datos](#)
- [Versionado Semántico - Especificación oficial](#)
- [Atlassian Git Tutorial - Tutoriales interactivos](#)
- [GitHub Flow - Flujo de trabajo colaborativo](#)



Gracias!

Módulo 8 Completado

Siguiente: Módulo 9 - Orquestación y automatización